Problem 1 (4 pts)

Consider the quadrature rule

$$\int_{-1}^{1} f(x)\,\mathrm{d}x \approx Af(-1) + Bf(0) + Cf(1).$$

It is given that the quadrature rule is exact when $f(x) = 1,\ x,\ x^2$ respectively.

(a) Find the values of $A,\ B,\ C$.

(b) If we approximate the integral $\int_{-1}^{1} f(x)\,\mathrm{d}x$ with this quadrature, is it true that the above approximation is exact if f is a polynomial of degree 3? Why? (Hint: let $f(x) = x^3$ and compute the values of $\int_{-1}^{1} f(x)\,\mathrm{d}x$ and the value given by the quadrature rule.)

(a) First let $f = 1$ then we get,

$$\begin{aligned}\int_{-1}^{1} 1\,\mathrm{d}x &= Af(-1) + Bf(0) + Cf(1)\\ \implies 2 &= A + B + C.\end{aligned}$$

Now let $f(x) = x$ and we get,

$$\begin{aligned}\int_{-1}^{1} x\,\mathrm{d}x &= A(-1) + B(0) + C(1)\\ \implies 0 &= -A + C.\end{aligned}$$

Finally let $f(x) = x^2$ and we get,

$$\begin{aligned}\int_{-1}^{1} x^2\,\mathrm{d}x &= A(-1)^2 + B(0)^2 + C(1)^2\\ \implies 2/3 &= A + C.\end{aligned}$$

This gives us the following system of equations.

$$\begin{cases} A + B + C = 2 \\ -A + 0 \cdot B + C = 0 \\ A + 0 \cdot B + C = 2/3 \end{cases}.$$

Solving the system we get, $A = 1/3,\ B = 4/3,\ C = 1/3$.

(b) Suppose that $\int_{-1}^{1} x^3\,\mathrm{d}x = A(-1)^3 + B(0)^3 + C(1)^3$. Computing we find that $-1A + C = 0$ must be true for equality to hold. Note that this is consistent with the system we found in part (a).

Therefore for $A = 1/3, B = 4/3, C = 1/3$, $\int_{-1}^{1} x^i \, \mathrm{d}x = A((-1)^i) + B(0^i) + C(1^i)$ holds true for $i = 0, 1, 2, 3$. Consider some arbitrary polynomial of degree 3, $f(x) = c_0 + c_1 x + c_2 x^2 + c_3 x^3$ and compute $\int_{-1}^{1} f(x) \, \mathrm{d}x$. We get,

$$\begin{aligned}
\int_{-1}^{1} f(x) \, \mathrm{d}x &= c_0 \int_{-1}^{1} 1 \, \mathrm{d}x + c_1 \int_{-1}^{1} x \, \mathrm{d}x + c_2 \int_{-1}^{1} x^2 \, \mathrm{d}x + c_3 \int_{-1}^{1} x^3 \, \mathrm{d}x \\
&= c_0 \left[A + B + C\right] + c_1 \left[-A + C\right] + c_2 \left[A + C\right] + c_3 \left[-A + C\right] \quad \text{(exact by part a)} \\
&= A \left(c_0 - c_1 + c_2 - c_3\right) + B \left(c_0 + 0 \cdot c_1 + 0 \cdot c_2 + 0 \cdot c_3\right) + C(c_0 + c_1 + c_2 + c_3) \\
&= Af(-1) + Bf(0) + Cf(1) \\
&= \frac{1}{3} f(-1) + \frac{4}{3} f(0) + \frac{1}{3} f(1) \quad \text{(using solution to system).}
\end{aligned}$$

Therefore for any polynomial of degree 3 or less $\int_{-1}^{1} f(x) \, \mathrm{d}x = \frac{1}{3} f(-1) + \frac{4}{3} f(0) + \frac{1}{3} f(1)$.

Problem 2 (6 pts)

Let $f(x)$ be a function defined on the interval $[-1, 1]$, and $f \in C^3([-1, 1])$.

(a) Let $h(x)$ be the Lagrange interpolation polynomial of $f(x)$ at the nodes $x = -1,\ 0,\ 1$. Write down the expression of $h(x)$.

(b) Compute the integral

$$\int_{-1}^{1} h(x) \, \mathrm{d}x$$

accurately in terms of the values of $f(x)$ at points $x = -1,\ 0,\ 1$.

(c) If we approximate the integral $\int_{-1}^{1} f(x) \, \mathrm{d}x$ by $\int_{-1}^{1} h(x) \, \mathrm{d}x$, is it true that the above approximation is exact if f is a polynomial of degree less than or equal to 2? Why?

(a) Using the nodes $x = -1, 0, 1$ we get following Lagrange polynomial,

$$\begin{aligned}
h(x) &= f(-1) \frac{(x-0)(x-1)}{(-1-0)(-1-1)} + f(0) \frac{(x+1)(x-1)}{(0+1)(0-1)} + f(1) \frac{(x+1)(x-0)}{(1+1)(1-0)} \\
&= \frac{f(-1)}{2}(x^2 - x) + \frac{f(0)}{-1}(x^2 - 1) + \frac{f(1)}{2}(x^2 + x) \\
&= \underbrace{\left(\frac{f(-1)}{2} - f(0) + \frac{f(1)}{2}\right)}_{A} x^2 + \underbrace{\left(\frac{f(1)}{2} - \frac{f(-1)}{2}\right)}_{B} x + f(0) \\
&= Ax^2 + Bx + f(0).
\end{aligned}$$

(b) We compute the integral.

$$\begin{aligned}\int_{-1}^{1} h(x)\,\mathrm{d}x &= A\int_{-1}^{1} x^2\,\mathrm{d}x + B\int_{-1}^{1} x\,\mathrm{d}x + \int_{-1}^{1} f(0)\,\mathrm{d}x \\ &= \frac{2}{3}A + B\cdot 0 + 2f(0) \\ &= \frac{1}{3}\left[f(-1) + 4f(0) + f(1)\right]\end{aligned}$$

(c) Note that the result we got from part (b) is actually Simpson's rule $\frac{b-a}{6}\left(f(a) + 4f(\frac{a+b}{2}) + f(b)\right)$, where the interval is $[-1, 1]$. We know Simpson's rule is exact for polynomials of degree 2 or less. Also true because we know the Lagrange polynomial with $n+1$ nodes of a polynomial of degree less than or equal to n is exact.

Problem 3 (4 pts)

(a) Prove the following theorem.

Theorem Let $q(x)$ be a nontrivial polynomial of degree $n+1$ such that, for all $0 \le k \le n$

$$\int_{-1}^{1} x^k q(x)\,\mathrm{d}x = 0.$$

Let $x_0, \ldots, x_n$ be the zeros of q. Then the following quadrature rule

$$\int_{-1}^{1} f(x)\,\mathrm{d}x \approx \sum_{i=0}^{n} A_i f(x_i)$$

with $A_i = \int_{-1}^{1} \frac{\prod_{j\neq i}(t-x_j)}{\prod_{j\neq i}(x_i-x_j)}\,\mathrm{d}t$ is exact for all polynomials of degree at most $2n+1$.
(Hint: given any polynomial f of degree $\le 2n+1$, by the division algorithm, we have

$$f(x) = p(x)q(x) + r(x)$$

where the degree of r is $\le n$.)

(b) Given that $q(x) = \frac{1}{2}(3x^2 - 1)$ is such that

$$\int_{-1}^{1} q(x)\,\mathrm{d}x = 0 \quad \text{and} \quad \int_{-1}^{1} xq(x)\,\mathrm{d}x = 0.$$

Apply the above theorem to show the quadrature rule

$$\int_{-1}^{1} f(x)\,\mathrm{d}x \approx f\left(-\frac{\sqrt{3}}{3}\right) + f\left(\frac{\sqrt{3}}{3}\right)$$

is exact up to a polynomial of degree 3.

(a) *Proof.* Let $f \in \mathcal{P}_{2n+1}(\mathbb{R})$ and dividing by the given $q(x) \in \mathcal{P}_{n+1}(\mathbb{R})$ we get,

$$f(x) = p(x)q(x) + r(x)$$

where $r(x) \in \mathcal{P}_n(\mathbb{R})$. Suppose we can write $p(x) = a_0 + a_1 x + \cdots + a_n x^n$ and compute $\int_{-1}^{1} f(x)\,\mathrm{d}x$. We get

$$\begin{aligned}
\int_{-1}^{1} f(x)\,\mathrm{d}x &= \int_{-1}^{1} p(x)q(x) + \int_{-1}^{1} r(x)\,\mathrm{d}x \\
&= \int_{-1}^{1} (a_0 q(x) + a_1 x q(x) + \cdots + a_n x^n q(x)) + \int_{-1}^{1} r(x)\,\mathrm{d}x \\
&= \sum_{i=0}^{n} a_i \cancelto{0}{\int_{-1}^{1} x^i q(x)\,\mathrm{d}x} + \int_{-1}^{1} r(x)\,\mathrm{d}x \quad \text{(cancellation follows from assumption)} \\
&= \int_{-1}^{1} r(x)\,\mathrm{d}x
\end{aligned}$$

Now consider the Lagrange interpolation of $r(x)$, an n degree polynomial, using $n+1$ nodes $x_0, x_1, \ldots, x_n$. We get,

$$\begin{aligned}
r(t) &= r(x_0)\frac{\prod_{j\neq 0}(t - x_j)}{\prod_{j\neq 0}(x_0 - x_j)} + \cdots + r(x_n)\frac{\prod_{j\neq n}(t - x_j)}{\prod_{j\neq n}(x_n - x_j)} \\
\implies \int_{-1}^{1} r(t)\,\mathrm{d}t &= r(x_0)\int_{-1}^{1} \frac{\prod_{j\neq 0}(t - x_j)}{\prod_{j\neq 0}(x_0 - x_j)}\,\mathrm{d}t + \cdots + r(x_n)\int_{-1}^{1} \frac{\prod_{j\neq n}(t - x_j)}{\prod_{j\neq n}(x_n - x_j)}\,\mathrm{d}t \\
&= \sum_{i=0}^{n} A_i r(x_i)
\end{aligned}$$

Finally note that for the given nodes x_i, $f(x_i) = p(x_i)q(x_i) + r(x_i) = r(x_i)$ by assumption that these nodes are zeros of $q(x)$. Thus we have that,

$$\int_{-1}^{1} f(x)\,\mathrm{d}x = \sum_{i=0}^{n} A_i r(x_i) = \sum_{i=0}^{n} A_i f(x_i)$$

□

(b) We note firstly that the given $q(x)$ has roots at $x_0 = -\frac{\sqrt{3}}{3}$ and $x_1 = \frac{\sqrt{3}}{3}$. We also note that q is of degree $n+1 = 2$ and thus $2n+1 = 3$. Thus by application of the theorem we have that for any arbitrary polynomial $f(x)$ of degree 3 or less,

$$\int_{-1}^{1} f(x)\,\mathrm{d}x = A_0 f\left(-\frac{\sqrt{3}}{3}\right) + A_1 f\left(\frac{\sqrt{3}}{3}\right)$$

Computing the A_i terms we find

$$\begin{aligned}
A_0 &= \int_{-1}^{1} \frac{(t - x_1)}{(x_0 - x_1)}\,\mathrm{d}t \\
A_0 &= \frac{-2x_1}{x_0 - x_1} \\
A_0 &= \frac{-2(\sqrt{3}/3)}{-2(\sqrt{3}/3)} = 1; \\
A_1 &= \int_{-1}^{1} \frac{(t - x_0)}{(x_1 - x_0)}\,\mathrm{d}t, \\
A_1 &= \frac{-2x_0}{x_1 - x_0} \\
A_1 &= 1. \quad \text{(plugging in for the nodes)}
\end{aligned}$$

Thus we have for any arbitrary polynomial of degree three or less,

$$\int_{-1}^{1} f(x)\,\mathrm{d}x = 1 \cdot f\left(-\frac{\sqrt{3}}{3}\right) + 1 \cdot f\left(\frac{\sqrt{3}}{3}\right)$$

Problem 4 (3 pts)

Computational problem. Consider the integral:

$$\int_{0}^{\pi/2} \cos x\,\mathrm{d}x$$

(a) Write a program to use the composite trapezoid rule to approximate the above integral by dividing $[0, \pi/2]$ to N equal spaces. *Only* submit your code.

(b) Tabulate the corresponding approximate values by the composite trapezoid rule with $N = 10,\ 20,\ 30, 40\ , 50$ as below.

N	Approximate value
10	
20	
30	
40	
50	

(a) You can find the code in code appendix.

(b) You'll find the results from the approximation below. They clearly tend to 1.000 as N increases.

N	Approximate value
10	0.9979429863511016
20	0.9994859052452822
30	0.9997715264921545
40	0.9998714862196685
50	0.9999177519404725

(c*) Just for fun I decided to compare the quadrature rules (Trapezoid vs. Simpson's rule) and got the following graph. Code is found at code appendix and clearly we see that Simpson's method has a much faster convergence.

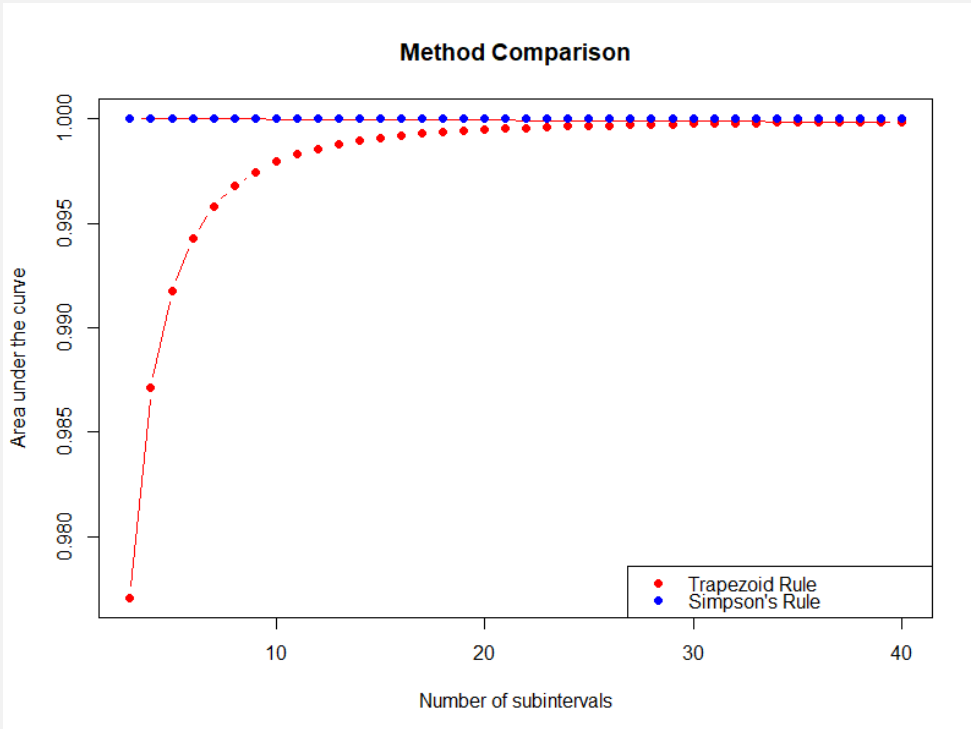

Figure 1: Comparing Convergence between Trapezoid and Simpson's Rules

Problem 5 (3 pts)

Computational problem. Consider

$$\begin{cases} y'(t) &= -15y \text{ on } [0,5] \\ y(0) &= 1 \end{cases}$$

(a) Use Euler's method to numerically approximate the solution $y(t)$ with a mesh size $1/4$. Plot the solution trajectory. Submit both you codes and graphs.

(b) Use Euler's method to numerically approximate the solution $y(t)$ with a mesh size $2/15$. Plot the solution trajectory. Submit both you codes and graphs.

(c) Use Euler's method to numerically approximate the solution $y(t)$ with a mesh size $1/100$. Plot the solution trajectory. Submit both you codes and graphs.

For all parts the code can be found in code appendix.

(a) For $h = 1/4$ we get,

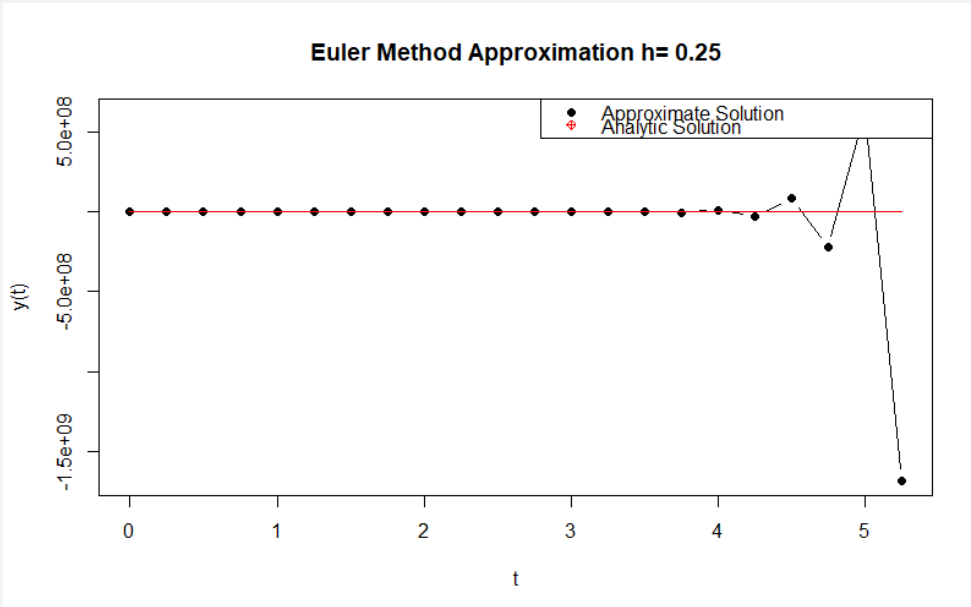

Figure 2: Euler Method Approximation Using $h = 1/4$

(b) For $h = 2/15$ we get,

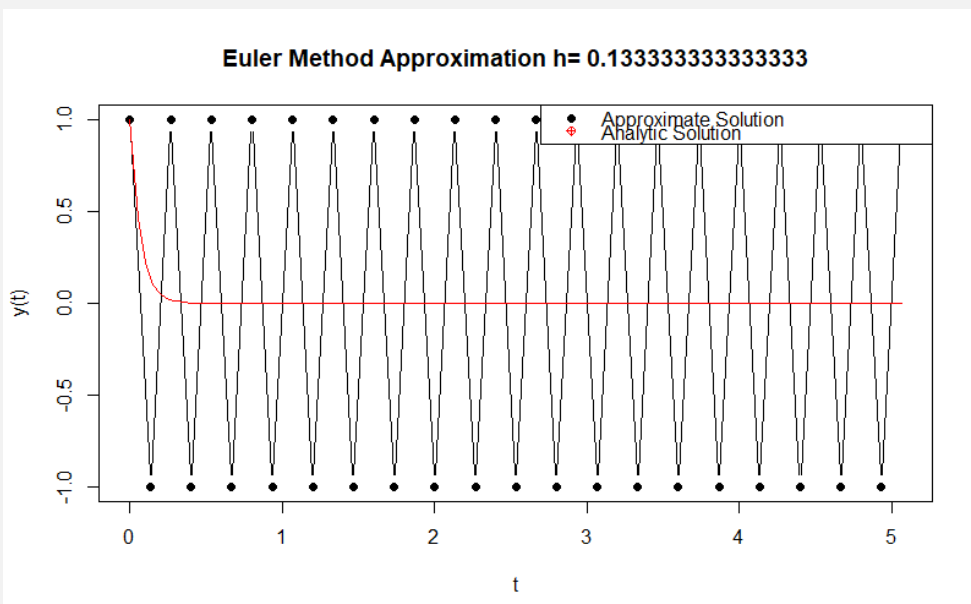

Figure 3: Euler Method Approximation Using $h = 2/15$

(c) For $h = 1/100$ we get,

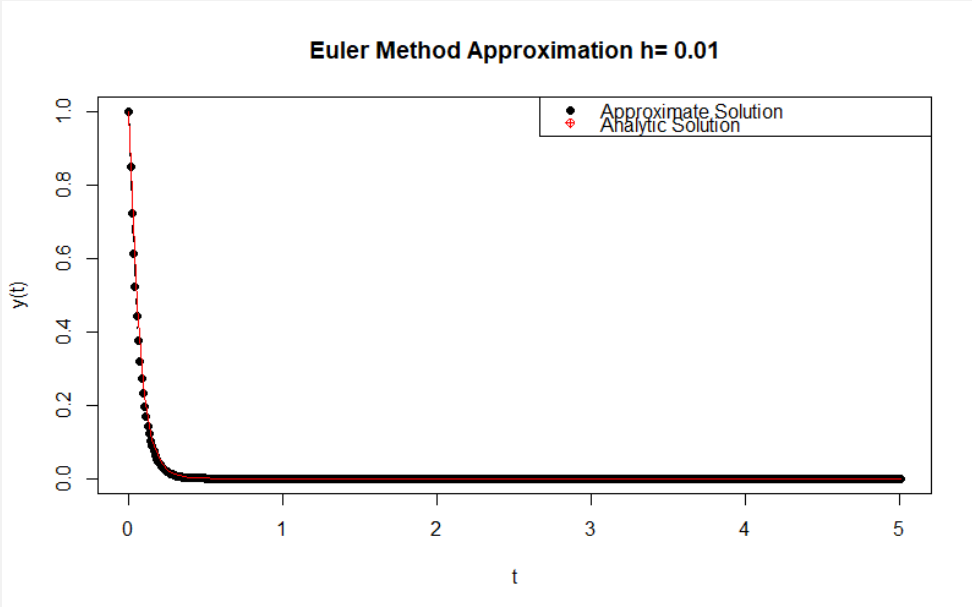

Figure 4: Euler Method Approximation Using $h = 1/100$

We note that we attain stability for a sufficiently small mesh size which is in accordance with what we know about Euler's Method.

Code Appendix

Algorithms are written in R.

Trapezoid Rule

```r
## Trapezoid Rule Algorithm ##

options(digits = 16)

## inputs: function f, interval [a,b], and number of subintervals N

trap_int <- function(f, a, b, N){
  if (b < a) {
    stop("Improper interval was input")
  }

  ## main interval
  A <- round(a, 10)
  B <- round(b, 10)

  ## all subintervals
  all_ints <- seq(A, B, length.out = N+1)

  ## trapezoid rule
  sum <- f(all_ints[1]) + f(all_ints[N+1])

  for (i in 2:(N)){
    sum <- sum + 2*f(all_ints[i])
  }

  sum <- sum*(b-a)/(2*N)
  ## output
  return(sum)
}


## Results for Class

for (i in seq(10,50,10)){
  print(trap_int(cos, 0, pi/2, i))
}
```

Euler's Method

```r
## Euler's Method


## Current Euler Method has some limitations
## Need to add:
## 1) general function input
## 2) more robust update loop

euler_method <- function(h, t0, y0){
```

```r

  y_old <- y0
  t_old <- t0

  l <- seq(0,5, by = h)

  y_seq <- c(y0)

  ## loop here
  for (i in l){
    y_new <- y_old + h * (-15*y_old)
    y_seq <- c(y_seq, y_new)
    y_old <- y_new
  }

  l <- c(l, l[length(l)]+h)

  plot(l,y_seq, type = "b", xlab="t", ylab = "y(t)",
       main =paste("Euler Method Approximation h=",h), pch = 19)
  curve(exp(-15*x), add = TRUE, col="red")
  legend("topright", legend = c("Approximate Solution", "Analytic Solution"),
         col = c("black", "red"), pch = c(19,10))


}

## outputs for homework
euler_method(0.25,0,1)
euler_method(2/15,0,1)
euler_method(1/100,0,1)
```

Simpson's Rule Comparison

```r
## Adding Simpson's Rule as well / pretty easy addition

simpson_int <- function(f, a, b, N){
  if (b < a) {
    stop("Improper interval was input")
  }

  ## main interval
  A <- round(a, 10)
  B <- round(b, 10)

  ## all subintervals
  all_ints <- seq(A, B, length.out = N+1)

  ## Simpson's rule
  sum <- f(all_ints[1]) + f(all_ints[N+1])

  for (i in 2:(N)){
    sum <- sum + 2*f(all_ints[i])
  }

  for (i in 1:N){
```

```r
    sum <- sum + 4*f(0.5*(all_ints[i]+all_ints[i+1]))
  }

  sum <- ((b-a)/(6*N))*sum
  ## output
  return(sum)
}

## Adding a Comparison of the Methods

Y1 <- c()
Y2 <- c()

for (i in (3:40)){
  Y1 <- c(Y1, trap_int(cos, 0, pi/2, i))
  Y2 <- c(Y2, simpson_int(cos, 0, pi/2, i))
}

plot(c(x,x),c(Y1,Y2), col = c(rep("red",38),rep("blue", 38)), type = "b",
     pch = 19, xlab = 'Number of subintervals', ylab = "Area under the curve",
     main = "Method Comparison")

legend("bottomright", legend = c("Trapezoid Rule", "Simpson's Rule"), col = c("red", "blue"),
    pch = 19)
```